

Introduction

- La sécurité d'un serveur web PHP est fragile puisqu'il permet d'accéder aux fichiers, d'exécuter des commandes, et d'ouvrir des connexions réseaux.
- La common gateway interface (CGI) est une interface de serveurs Web qui permet un échange de données normalisé entre des applications et des serveurs externes. Ainsi, les pages HTML ne sont pas entièrement disponibles sur le serveur HTTP qui exécute un autre programme, puis retourne le contenu généré.
- Utiliser PHP comme un exécutable CGI est une possibilité pour les cas où l'on ne veut pas l'utiliser comme un module du serveur web (comme Apache), ou bien lorsque l'on souhaite l'utiliser en combinaison avec un gestionnaire CGI complémentaire, afin de créer un environnement de script sécurisé.
- Lorsque PHP est utilisé en tant que module Apache, celui-ci hérite des permissions accordées à l'utilisateur faisant tourner Apache ce qui peut impacter la sécurité et les autorisations.
- Une erreur de sécurité fréquente est de donner à l'utilisateur Apache les droits de superadministrateur ("root"), ou d'accroître les possibilités d'Apache d'une quelconque autre façon.
- PHP, est constamment testé et amélioré. Chaque nouvelle version rassemble des modifications majeures ou mineures, aussi bien pour renforcer la sécurité, que pour réparer des problèmes de conception et de configuration, ainsi que d'autres points qui peuvent affecter la sécurité et la stabilité globale de votre système.

Sécurité des sessions

- Pour protéger les utilisateurs d'une tactique simple, la directive `session.use_only_cookies` doit être activée. Dans ce cas, les cookies doivent être activés obligatoirement côté client sinon les sessions ne fonctionneront pas.
- Implémenter SSL/TLS sur le serveur et le rendre obligatoire pour les utilisateurs. HSTS devrait être utilisé pour améliorer également la sécurité.
- Activer `session.use_strict_mode` quand le gestionnaire de sauvegarde des sessions le supporte, ce qui permet de rejeter un identifiant de session non initialisé, et d'en créer un nouveau.
- Utiliser les timestamps basés sur le gestionnaire de session, et régénérer les identifiants de session en utilisant la fonction `session_regenerate_id()` avec les procédures recommandées.
- `session_regenerate_id()` n'effacera pas les sessions anciennes par défaut. Si un utilisateur accède à une session obsolète (session ayant expirée), l'accès à cette session doit être refusé en utilisant des timestamps. Il est également recommandé de supprimer le statut d'authentification de toutes les sessions utilisateurs sinon cela peut représenter un axe d'attaque.
- Utiliser la fonction `session_gc()` pour supprimer les données obsolètes depuis le stockage des données de sessions.
- Utiliser l'option 'read_and_close' avec la fonction `session_start()`. `session_start(['read_and_close'=>1]);` va clôturer la session aussi vite que possible après la mise à jour de la variable globale `$_SESSION` en utilisant la fonction `session_commit()`.
- N'utilisez jamais de données confidentielles comme préfixe. Si l'identifiant utilisateur est confidentiel, vous devriez utiliser la fonction `hash_hmac()`.
- Utiliser une clé de hachage sécurisé à usage unique comme clé d'auto-identification en utilisant la fonction `setcookie()`. Utiliser un hachage sécurisé plus fort que SHA-2. i.e. SHA-256 ou supérieur avec des données aléatoires depuis la fonction `random_bytes()` ou via `/dev/urandom`.
- Utiliser la fonction `output_add_rewrite_var()` pour la protection des fraudes de requêtes intersites.

Sécurisation des configurations INI de session

- **session.cookie_lifetime=0** : Elle informe les navigateurs de ne pas stocker le cookie dans un espace de stockage permanent. Aussi, lorsque le navigateur se ferme, le cookie d'identification de session est supprimé immédiatement.
- **session.use_cookies=On et session.use_only_cookies=On** : Si session.use_only_cookies=Off, le module de session utilisera les valeurs de l'identifiant de sessions définies par les variables GET/POST/URL fournies, et le cookie de l'identifiant de session ne sera pas initialisé.
- **session.use_strict_mode=On** : Ce mode évite que le module de session utilise un identifiant de session non initialisé
- **session.cookie_httponly=On** : Permet de refuser l'accès à un cookie de session depuis javascript. Cette configuration évite qu'un cookie ne soit corrompu par une injection Javascript.
- **session.cookie_secure=On** : Permet d'accéder au cookie d'identifiant de session uniquement lorsque le protocole est HTTPS.
- **session.cookie_samesite="Lax" ou session.cookie_samesite="Strict"** : Les cookies utilisant Lax seront accessibles via une requête GET originaire d'un autre domaine, alors que les cookies utilisant Strict ne le seront pas. Cet attribut est une façon de mitiger les attaques de fraudes de requêtes intersites.
- **session.gc_maxlifetime=[choisissez le plus petit possible]** : est une configuration pour supprimer l'identifiant de session obsolète.
- **session.use_trans_sid=Off** : La désactivation de la gestion des identifiants de session de façon transparente permet de sécuriser un peu plus les identifiants de session en éliminant la possibilité d'une injection d'identifiant de session ou de fuite de cet identifiant.
- **session.trans_sid_tags=[drapeaux limités]** : (PHP 7.1.0 >=) Les développeurs ne doivent pas réécrire de drapeaux HTML non nécessaires. La valeur par défaut doit être suffisante pour la plupart des utilisations.

Sécurisation des configurations INI de session (suite)

- **session.trans_sid_hosts=[hôtes limités]** : (PHP 7.1.0 >=) Ce paramètre définit une liste blanche des hôtes qui sont autorisés à réécrire les identifiants de session transparents.
- **session.referer_check=[URL d'origine]** : Lorsque le paramètre session.use_trans_sid est actif. Ce paramètre réduit les risques d'injection d'identifiant de session.
- **session.cache_limiter=nocache** : S'assure que le contenu HTTP n'est pas mis en cache pour les sessions authentifiées. Permet la mise en cache que pour les contenus qui ne sont pas privés.
- **session.sid_length="48"** : (PHP 7.1.0 >=) La longueur des identifiants de session permet d'avoir des identifiants de session plus forts. Les développeurs doivent utiliser des identifiants de session d'au moins 32 caractères. Au moins 26 caractères sont requis lorsque session.sid_bits_per_character="5".
- **session.sid_bits_per_character="6"** : (PHP 7.1.0 >=) Le nombre d'octets présents dans un caractère d'identifiant de session.
- **session.hash_function="sha256"** : (PHP 7.1.0 <) Une fonction de hachage forte va générer un identifiant de session fort.
- **session.save_path=[dossier non lisible par tout le monde]** : Si ce paramètre est défini à un dossier accessible en lecture par tout le monde, comme /tmp (par défaut), les autres utilisateurs du serveur seront capables de récupérer les sessions en listant les fichiers présents dans ce répertoire.

Sécurité des fichiers

- Puisque PHP a été fait pour permettre aux utilisateurs d'accéder aux fichiers, il est possible de créer un script PHP qui vous permet de lire des fichiers tels que `/etc/password`, de modifier les connexions ethernet, lancer des impressions de documents, etc.
- Il y a deux mesures primordiales à prendre pour éviter ces manœuvres :
 - Limiter les permissions de l'utilisateur web PHP.
 - Vérifier toutes les variables liées aux chemins et aux fichiers qui sont fournis.
- Si votre système d'identification permet aux utilisateurs de créer leur propre login, et qu'un utilisateur choisi le login `../etc/`, le système est de nouveau exposé. Suivant votre système d'exploitation, vous devrez protéger un grand nombre de fichiers, notamment les entrées de périphériques, (`/dev/` ou `COM1`), les fichiers de configuration (fichiers `/etc/` et `.ini`), les lieux de stockage d'informations (`/home/`, `My Documents`), etc. Pour cette raison, il est généralement plus sûr d'établir une politique qui interdit TOUT sauf ce que vous autorisez.

Sécurité des bases de données

- Pour lire ou stocker des informations, vous devez vous connecter au serveur de bases de données, envoyer une requête valide, lire le résultat et refermer la connexion. De nos jours, le langage le plus courant pour ce type de communication est le langage SQL. Cela signifie qu'une requête SQL est capable de contourner les contrôles et vérifications, comme les identifications, et parfois, les requêtes SQL ont accès aux commandes d'administration.
- L'injection SQL directe est une technique où un pirate modifie une requête SQL existante pour afficher des données cachées, ou pour écraser des valeurs importantes, ou encore exécuter des commandes dangereuses pour la base. Ces attaques sont généralement basées sur l'exploitation de code qui n'est pas écrit de manière sécuritaire.
- Ne vous connectez jamais sur une base de données en tant que super utilisateur ou propriétaire de la base. Utilisez toujours un utilisateur adapté, avec des droits très limités. N'affichez jamais d'informations spécifiques à la base, et notamment des informations concernant le schéma.
- Utilisez des requêtes préparées avec des variables liées. Elles sont disponibles avec PDO, MySQLi ainsi que d'autres bibliothèques.
- Vérifiez que les données ont bien le type attendu. Si l'application attend une entrée numérique, vérifiez vos données avec la fonction `ctype_digit()`, ou bien modifiez automatiquement le type avec la fonction `settype()`, ou encore avec `sprintf()`.
- Si la couche de base de données ne suppose pas les variables liées, alors, mettez entre guillemets toutes les valeurs non numériques qui sont passées à la base de données avec la fonction spécifique à la base de données d'échappement de caractères (e.g. `mysql_real_escape_string()`, `sqlite_escape_string()`, etc.).
- Il est recommandé d'établir des connexions au serveur avec le protocole SSL, pour chiffrer les échanges clients/serveur, afin d'améliorer la sécurité. Vous pouvez aussi utiliser un client SSH pour chiffrer la connexion entre les clients et le serveur de bases de données.
- Dans le cas de données vraiment sensibles, si la représentation originale n'est pas nécessaire (pour affichage, ou comparaison), utiliser un hash est une bonne solution.

Sécurité des données transmises par les internautes

- Il est vivement recommandé d'examiner minutieusement votre code pour vous assurer qu'il n'y a pas de variable envoyée par le client web qui ne soit pas suffisamment vérifiée avant utilisation.
- Il est conseillé de répondre aux questions suivantes :
 - Est-ce que ce script n'affectera que les fichiers prévus ?
 - Est-il possible que des valeurs incohérentes ou inattendues soient exploitées ici ?
 - Est-ce que ce script peut être utilisé dans un but différent de celui attendu ?
 - Est-ce que ce script peut être utilisé malicieusement, en conjonction avec d'autres ?
 - Est-ce que toutes les actions seront correctement historisées ?
- Envisager de supprimer l'acquisition automatique des variables (register_globals), ou encore toute option qui pourrait conduire à vous tromper sur la validité, la source, ou la valeur d'une variable.
- Travailler avec PHP en mode error_reporting(E_ALL), peut aussi vous aider à détecter que certaines variables sont utilisées avant d'être exploitées ou initialisées.